

پروژه UniDB

طراحی موتور دیتابیس NoSQL

فاز دوم

ساختمان داده‌ها

دکتر افسانه فاطمی

زمستان ۱۴۰۴

تاریخ تحویل:

۱۴۰۴/۱۱/۰۳

مباحث تحت پوشش

درخت‌های جستجو (BST/AVL)، جداول درهم‌سازی (Hash)، ایندکس معکوس و بهینه‌سازی کوئری

مقدمه

در فاز اول، شما موتور ذخیره‌سازی را با استفاده از لیست پیوندی و آرایه پیاده‌سازی کردید. مشکل اصلی آن طرح این بود که برای پیدا کردن یک رکورد خاص، باید تمام داده‌ها را پیمایش می‌کردید (Scan خطی با $O(n)$). این روش برای ۵۰,۰۰۰ رکورد شاید جواب دهد، اما برای داده‌های میلیونی بسیار کند است.

در فاز دوم، ما قابلیت ایندکس‌گذاری (Indexing) را به UniDB اضافه می‌کنیم تا سرعت جستجو را به $O(\log n)$ یا حتی $O(1)$ برسانیم.

نیازمندی‌های جدید: انواع ایندکس‌ها

۱. ایندکس درخت جستجوی دودویی (BST Index)

- کاربرد: برای جستجو روی فیلدهای عددی و رشته‌ای (مثل `id`).
- عملیات: `insert`, `delete`, `search` باید پیاده‌سازی شوند.
- چالش: درخت ممکن است نامتوازن شود و در بدترین حالت به $O(n)$ میل کند.

۲. ایندکس درخت متوازن (AVL Tree Index)

- کاربرد: همانند BST، اما با تضمین پیچیدگی $O(\log n)$.
- چالش: پیاده‌سازی عملیات‌های `Rotation` (چرخش چپ و راست) هنگام درج و حذف برای حفظ تعادل درخت.

۳. ایندکس جدول درهم‌سازی (Hash Index)

- کاربرد: جستجوی دقیق (Exact Match) بسیار سریع $O(1)$ روی فیلدهایی مثل `id` یا `.national_code`.
- روش برخورد: از روش زنجیره‌ای (Chaining) با استفاده از لیست پیوندی فاز ۱ استفاده کنید.
- تابع Hash: یک تابع هاش مناسب برای رشته‌ها و اعداد طراحی کنید.

۴. ایندکس معکوس (Inverted Index) - ویژه جستجوی

متنی

- کاربرد: جستجوی کلمات داخل متن (Full Text Search). مثلاً پیدا کردن تمام دانشجویانی که در نامشان کلمه "Ali" دارند.

- ساختار: یک Hash Map یا Tree که در آن:

★ کلید: کلمات موجود (Token ها).

★ مقدار: لیستی از ID های اسنادی که این کلمه در آن‌ها آمده است.

مثال: اگر نام دانشجو "Ali Rezaei" باشد، ایندکس باید هم برای "Ali" و هم برای "Rezaei" به این دانشجو اشاره کند.

تغییرات در موتور اجرا (Execution Engine) ⚙️

موتور اجرای شما باید هوشمند شود. وقتی کوئری **filter** می‌آید، نباید بلافاصله اسکن خطی کند.

💡 منطق انتخاب مسیر (Query Optimizer):

بررسی کنید آیا روی فیلد خواسته شده، ایندکسی وجود دارد؟

- اگر بله: از ایندکس استفاده کنید (Search در درخت یا Hash).

- اگر خیر: کل کالکشن را پیمایش کنید (Full Scan - روش فاز ۱)

دستورات جدید:

```
// ساخت ایندکس روی یک فیلد خاص با الگوریتم مشخص
unidb> db.students.createIndex("gpa", "bst")
unidb> db.students.createIndex("id", "hash")
unidb> db.students.createIndex("name", "inverted")

// جستجو (موتور خودکار تشخیص می‌دهد از کدام ایندکس استفاده کند)
unidb> db.students.filter("id", 101)           // Uses Hash Index -> O(1)
unidb> db.students.filter("gpa", 19.5)         // Uses BST Index -> O(log n)
unidb> db.students.search("name", "Ali")       // Uses Inverted Index (Full Text)
```

همگام‌سازی (Synchronization)

بزرگترین چالش این فاز، هماهنگ نگه داشتن ایندکس‌ها با داده‌های اصلی است.

- هرگاه **insert** انجام می‌شود، باید در تمام ایندکس‌های فعال هم درج انجام شود.
- هرگاه **delete** انجام می‌شود، باید از تمام ایندکس‌ها هم حذف شود.

⚠ **توجه:** اگر حذف از AVL را پیاده‌سازی نکردید، نمره آن بخش کسر می‌شود اما کل پروژه را متوقف نکنید.

✨ مثال کامل تعامل (Interaction Scenario)

سناریوی زیر نشان می‌دهد چطور کاربر از ویژگی‌های فاز ۲ استفاده می‌کند:

```
// 1. Initial State (No Indexes)
// This query is SLOW because it scans all records
unidb> db.students.filter("id", 10245)
Result: Found 1 record (Time: 50ms, Scanned: 50000)

// 2. Create Indexes
unidb> db.students.createIndex("id", "hash")
unidb> db.students.createIndex("gpa", "avl")
unidb> db.students.createIndex("name", "inverted")

// 3. Fast Queries (Using Indexes)
// Now the same query uses Hash Map -> O(1)
unidb> db.students.filter("id", 10245)
Result: Found 1 record (Time: 0.1ms, Scanned: 1)

// Search for range (using AVL) -> O(log n)
// Note: You need to implement range filter logic in parser
unidb> db.students.filter("gpa", 18.5)

// 4. Full Text Search
// Finds "Ali Rezaei", "Mohammad Ali", etc.
unidb> db.students.search("name", "Ali")
Result: Found 15 records.

// 5. Automatic Updates
// Inserting a new record updates ALL 3 indexes automatically
unidb> db.students.insertOne({_id: 999, name: "New Student", gpa: 20})
// If you query now, "New Student" is immediately searchable via hash, avl, and inverted index.
```

بخش تحلیلی و گزارش

۱. مقایسه AVL و BST

- داده‌هایی را به صورت مرتب (Sorted) وارد کنید (مثلاً اعداد ۱ تا ۱۰,۰۰۰).
- زمان جستجو در BST معمولی و AVL را مقایسه کنید. (در BST باید بسیار کند باشد چون شبیه لیست می‌شود).

۲. تاثیر Hash

- زمان جستجوی **findByID** را در سه حالت مقایسه کنید:
 ۱. بدون ایندکس ($Scan(O(n))$)
 ۲. با ایندکس ($BST(O(\log n))$)
 ۳. با ایندکس ($Hash(O(1))$)

۳. تحلیل Inverted Index

- توضیح دهید چرا برای پیدا کردن کلمه "Ali" در نام‌ها، استفاده از **Inverted Index** بسیار سریع‌تر از **substring search** روی تک‌تک رکوردهاست؟

⚠ **توجه:** پاسخ این بخش می‌بایست به صورت فایل pdf در کنار سایر فایل‌های پروژه، روی ریپوزیتوری‌تان قرار بگیرد.

قوانین کلین کد (Clean Code Rules)

(کسر نمره تا ۲۰۰ - نمره)

این بخش اجباری است و رعایت نکردن آن باعث کسر شدید نمره می‌شود. هدف این است که پروژه از یک کد تمرینی ساده تبدیل شود به کدی قابل نگهداری، معماری‌مند و حرفه‌ای.

✨ موارد مورد انتظار:

- پیاده‌سازی هر ساختمان داده در کلاس جداگانه (BST، AVL، Hash، Inverted Index)
- تفکیک منطق Index، Storage و Query از یکدیگر
- نام‌گذاری معنادار و دقیق متغیرها، متدها و کلاس‌ها
- کوتاه و تک‌مسئولیتی بودن متدها
- جلوگیری از کد تکراری در عملیات مشترک (Insert / Delete / Search)
- استفاده صحیح از Interface یا کلاس پایه برای ایندکس‌ها
- خوانایی کد بدون نیاز به توضیح شفاهی

⚠️ **توجه:** حتی اگر تمام بخش‌های فنی پروژه کامل پیاده‌سازی شده باشند، عدم رعایت اصول Clean Code می‌تواند باعث **نمره منفی** شود.

بارمبندی پروژه (۲۰۰۰ نمره)

۱. ۷۰۰ نمره - پیاده‌سازی درخت‌ها (Tree Indexes)

- ۳۰۰ نمره: BST (درج، حذف و جستجو)
- ۴۰۰ نمره: AVL (درج با حفظ تعادل)

۲. ۴۰۰ نمره - پیاده‌سازی Hash Index

- طراحی تابع Hash و مدیریت برخورد (Collision)

۳. ۴۰۰ نمره - پیاده‌سازی Inverted Index

- توکن‌بندی رشته‌ها و ساخت لیست اسناد

۴. ۳۰۰ نمره - یکپارچه‌سازی (Integration)

- ۱۵۰ نمره: آپدیت خودکار ایندکس‌ها هنگام Insert/Delete
- ۱۵۰ نمره: استفاده هوشمند از ایندکس در کوئری‌ها (Optimizer)

۵. ۲۰۰ نمره - گزارش فنی

- نمودارهای مقایسه‌ای و تحلیل «داده‌های مرتب» در AVL vs BST

۶. Clean Code – (۲۰۰-)

- پیاده‌سازی تمام منطق در یک کلاس یا فایل
- متدهای طولانی و چندمنظوره
- نام‌گذاری مبهم یا غیرمرتبط
- وابستگی شدید بین ساختمان داده‌ها
- کد اسپاگتی یا پیاده‌سازی غیرقابل توسعه

